RISC-V ISA

Table des matières

1 Objectifs **1**

2 Instructions **3**
- 2.1 Constantes (Immediate) . . . 4
- 2.2 Calculs de base . . . 4
- 2.3 Accès mémoire . . . 4
- 2.4 Algorithmes . . . 4

3 Programmation impérative **6**
- 3.1 Setup Ripes . . . 6
- 3.2 Branching . . . 7
- 3.3 Loops . . . 8
- 3.4 Functions . . . 8
- 3.5 Algorithms . . . 9
  - 3.5.1 Fibonacci par récursivité . . . 9
  - 3.5.2 Modulo . . . 10
  - 3.5.3 Nombre carré . . . 10
  - 3.5.4 °F -> °C . . . 10

4 Assembler / Disassembler **12**
- 4.1 `HEIRV32-asm` Commandes . . . 12
  - 4.1.1 Assembleur vers binaire . . . 12
  - 4.1.2 Binaire vers assembleur . . . 13
  - 4.1.3 Prise en main . . . 14
- 4.2 Fonction main . . . 14
- 4.3 Reverse Engineering . . . 15

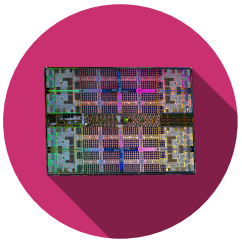
Acronymes **18**

1 Objectifs

Ce laboratoire est divisé en plusieurs blocs réalisés sur plusieurs semaines. Le but est de se familiariser avec le langage assembleur du RISC-V.

- La partie 2 Instructions du laboratoire porte sur les instructions individuelles.
- La partie 3 Programmation impérative du laboratoire nous amène à la programmation impérative, respectivement aux boucles et aux fonctions.

- La dernière partie 4 Assembler / Disassembler du laboratoire est consacrée au travail sur le compilateur : l'assemblage et le désassemblage d'un programme (passage d'un langage haut-niveau à du code machine et inversement).



2 Instructions

Dans cette première partie du laboratoire, nous allons travailler avec l'interpréteur RISC-V sur le site https ://hei-isc-car.github.io/riscv-interpreter/, voir illustration 1. Celui-ci permet aux registres d'écrire en mémoire et d'exécuter le code étape par étape. Ces outils en ligne vous aideront à résoudre et à contrôler les tâches.

Seules les instructions listées sur cette page sous **Supported Instructions** peuvent être utilisées. Les pseudo-instructions ne sont pas prises en charge.

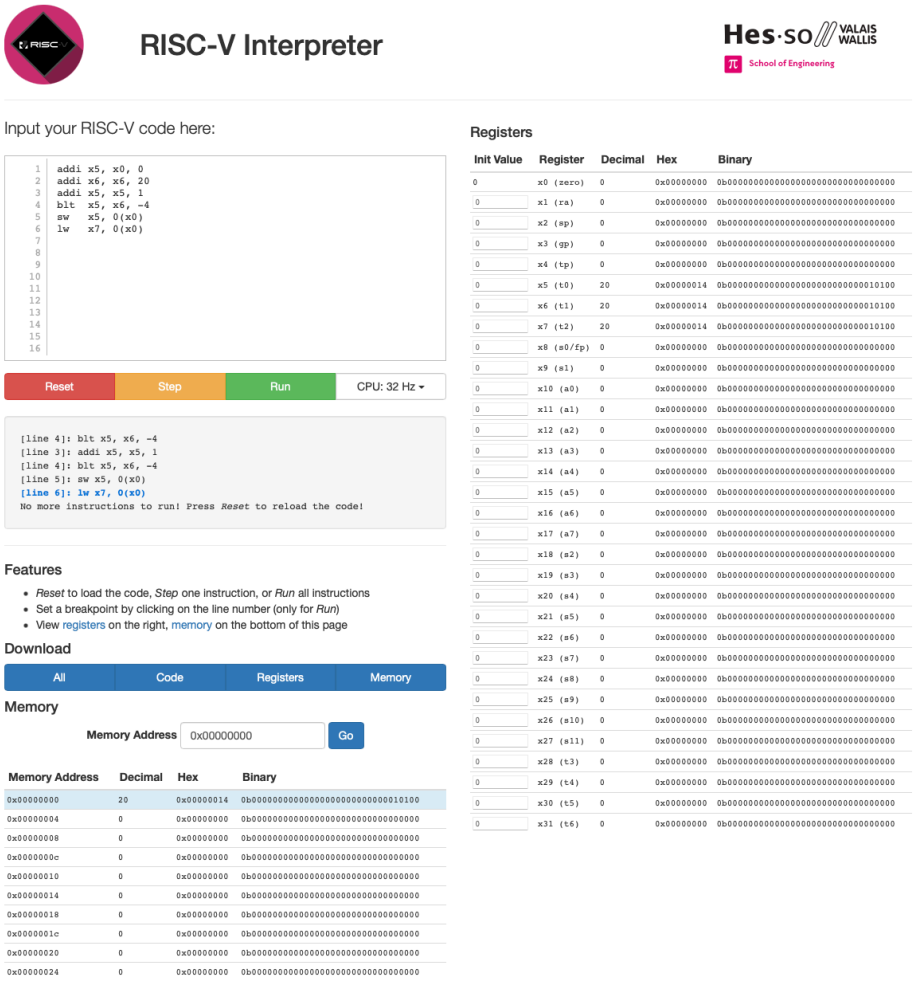


FIGURE 1 – Interpréteur RISC-V en ligne

Attention aux types des variables !

- Le type **int** est considéré de taille 32 bits signé.
- Le type **unsigned int** est considéré de taille 32 bits non-signé.
- Si il est suivi d'un nombre (ex : **int16_t**), cela signifie que la variable est sur x bits (ici 16). Si précédé d'un **u**, il est non-signé.

uint8_t est donc un byte non-signé, tandis que **int8_t** est un byte signé.

2.1 Constantes (Immediate)

Écrire le code assembleur RV32i pour les instructions suivantes :

a)
```
int a = 10;
int b = 0;
a = a + 4;
b = a - 12;

int i = 0;
int x = 2032;
int y = -78;
```

b)
```
int a = 0xABCDE123;

int b = 0xFEEDA987;
```

2.2 Calculs de base

Écrire le code assembleur RV32i pour les instructions suivantes :

a)
```
int b = 1;
int c = 2;
a = b + c;

int b = -1;
int c = 2;
a = b + c;

int b = -12;
int c = 2023;
a = b + c;
```

b)
```
int b = 1;
int c = 2;
int d = 5;
a = b + c - d;

int b = -1;
int c = 2;
int d = -78;
a = b + c - d;

int b = -12;
int c = 2023;
int d = 22;
a = b + c - d;
```

c)
```
int b = 2;
int c = 3;
int e = -1;
int f = -78;
int g = 2023;
int h = -12;
a = b - c;
d = (e + f) - (g + h);
```

2.3 Accès mémoire

Écrire le code assembleur RV32i pour les instructions suivantes :

a)
```
int a = mem[2];

int b = mem[8375];
```

b)
```
mem[5] = 42;
```

c)
```
uint16_t a = mem[3];
mem[4] = a;

int16_t a = mem[3];
mem[4] = a;
```

2.4 Algorithmes

Écrire le code assembleur RV32i pour les algorithmes suivants, en utilisant uniquement les instructions de base sans *loops*. *conditionals* ainsi que *branches* :

a) Dans un système, il peut être nécessaire d'attendre sans rien faire, sans modifier l'état actuel du système (pas d'accès mémoire, pas de modification des registres). Cette opération est communément appelée **NOP** (**NO O**peration). Proposez une instruction pour ce faire.

b) Calculer les 10 premiers nombres de Fibonacci. Enregistre chaque nombre dans un registre différent.

c) Transmettre la valeur de 8 bits de la mémoire à l'adresse $0x0000'1000$ en série, bit par bit, dans le LSB de la mémoire à l'adresse $0x0000'1001$. Les bits restants de l'adresse mémoire $0x0000'1001$ doivent être '0'. Calculez le débit de bauds en $\frac{Instructions}{Bit}$ pour l'ensemble de la transmission.

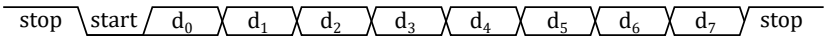


FIGURE 2 – UART Transmission en série

d) Multiplier deux nombres de 4 bits ensemble en utilisant en plus une des commandes bne, bge. L'algorithme fonctionne de la manière suivante : une multiplication est la même chose que l'addition x fois du même nombre. Par exemple : $2 * 9 = 9 + 9 = 18$.

3 Programmation impérative

Dans ce laboratoire, vous pouvez continuer à utiliser l'Interpréteur RISC-V en ligne. En outre, le programme **Ripes** est également disponible, voir l'illustration 3. Interface graphique de développement de Ripes. Vous devez d'abord le télécharger et le configurer.

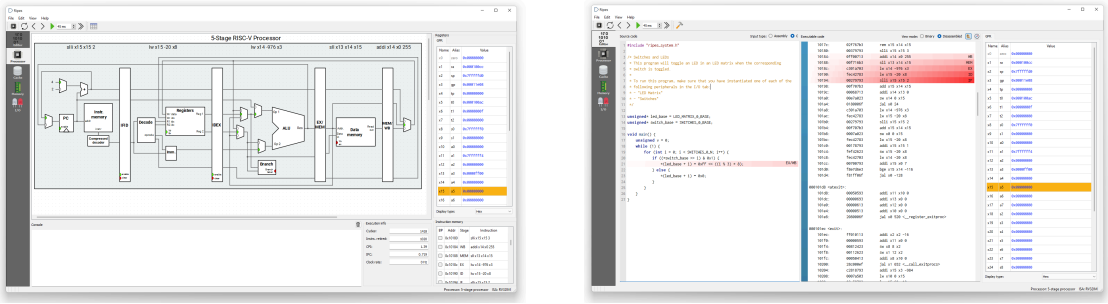


FIGURE 3 – Interface graphique de développement de Ripes

3.1 Setup Ripes

Sur les PCs de laboratoire, sous **C :/eda/RiscV** se trouvent Ripes et le compilateur **gcc** énoncé ci-après.

1. Téléchargez la dernière version du programme pour votre plateforme en suivant le lien : https ://github.com/mortbopet/Ripes/releases.
2. Téléchargez la version suivante de la chaîne d'outils RISC-V-GNU pour votre plateforme en suivant le lien : https ://github.com/sifive/freedom-tools/releases/tag/v2020.04.0-Toolchain.Only
3. Décompressez les deux et copiez le dossier RISC-V-GNU-Toolchain dans le dossier Ripes.
4. Lancez Ripes et configurez les paramètres du compilateur dans : *Edit* → *Settings*. Pour cela, vous devez sélectionner le fichier `bin/riscv64-unknown-elf-gcc-8.3.0.exe`.

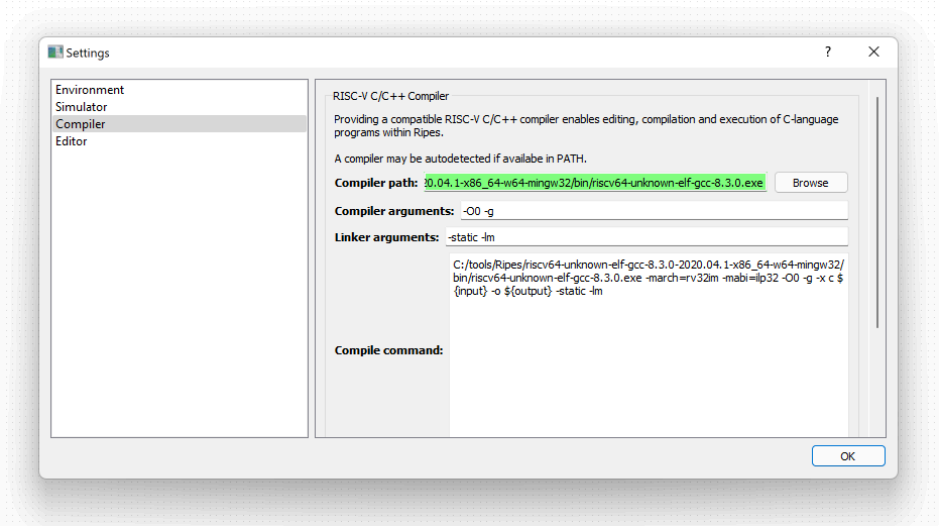


FIGURE 4 – Paramètres de la chaîne d'outils de Ripes

5. Dans les paramètres du processeur, sélectionnez le processeur monocycle RISC-V → 32-bit → sans extensions ISA.

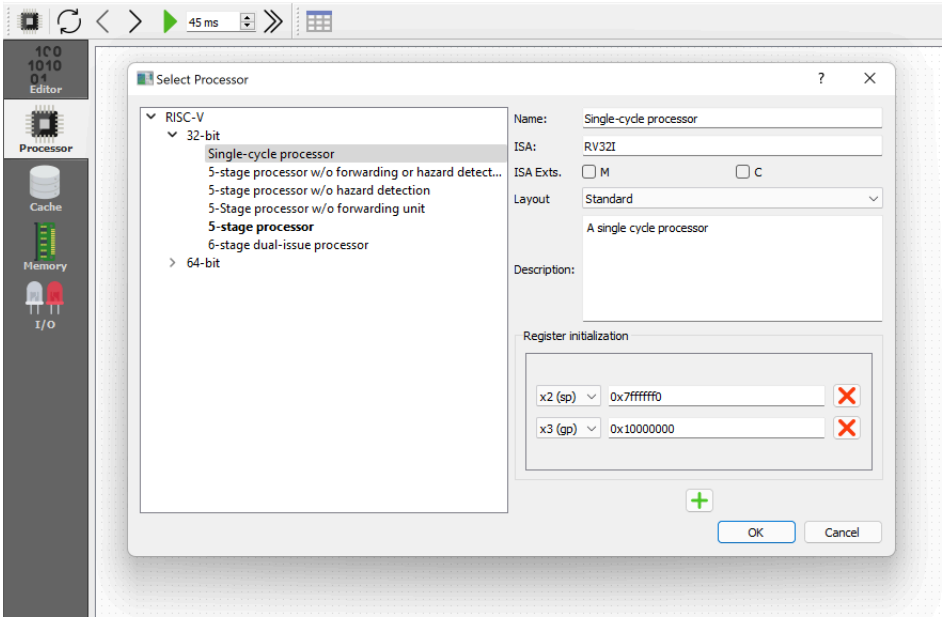


FIGURE 5 – Paramètres du processeur de Ripes

3.2 Branching

If / else

a)
```
int a = 1, b = 2, c;

if(a == b) {
  c = 1;
}
else {
  c = 0;
}
```

b)
```
int a = 1, b = 2, c;

if(a == b) {
  c = 0;
} else if(a > b) {
  c = 1;
} else {
  c = 2;
}
```

Switch case

a)
```
int a, b;

switch(b) {
  case 0:
    a = 17;
    break;
  default:
    a = 99;
}
```

b)
```
int a;

switch(mem[2]) {
  case 0:
    a = 17;
    break;
  case 3:
    a = 33;
    break;
  case 8:
  case 12:
    a = 10;
    break;
  default:
    a = 99;
}
```

3.3 Loops

While / Do While

a)
```
// A
int a = 10;

do{a = a - 1;}
while(a != 0);

// B
int a = 10;

while(a >= 0)
{a = a - 1;}
```

b)
```
// A
int a = 10;

do{a = a - 1;}
while(a != 0);

// B
int a = 10;

do{a = a - 1;}
while(a >= 0);

// C
unsigned int a = 10;

do{a = a - 1;}
while(a >= 0);
```

For

a)
```
unsigned int a = 0, i;

for(i = 0; i < mem[0]; i = i + 1) {
  a = a + 2;
}
```

b)
```
int a = 0, i;

for(i = 4; i > mem[0]; i = i - 1) {
  a = a + i;
}
```

c)
```
// An array of 10 bytes
uint8_t myArray[10] = ...
// ...

// Let say myArray[0] is at the
// address saved in register s0.
// Arrays in C are contiguous in memory.

int i;

for(i = 0; i < 10; i = i + 1) {
  myArray[i] = myArray[i] - 5;
}
```

3.4 Functions

Bien qu'un programmeur seul puisse agencer son code comme il l'entend, certaines conventions ont été établies pour permettre l'interopérabilité de plusieurs sources de code entre elles.

La fonction en appelant une autre est nommée **caller** et la fonction appelée nommée **callee**. Elles doivent être synchronisées sur la façon dont sont passés les arguments, quels registres doivent être conservés ...

Les notions suivantes sont les règles essentielles :

- Les registres **a0** à **a7** permettent de passer des arguments. Si plus sont nécessaires, ils sont passés sur le stack.
- Le résultat est placé dans le registre **a0**.
- Le **caller** enregistre l'addresse de retour (PC + 4) générée par l'instruction **jal** dans le registre **ra**.
- Le **callee** ne doit pas réécrire l'adresse de retour, le stack ou encore les registres sauvés sXX. Si elles sont utilisées, l'espace doit être réservé sur le stack et ces informations doivent être stockées puis restaurées.

a)
```
doNothing();

...

void doNothing() {
  return;
}
```

b)
```
int a = 1, b;
b = doubleIt(a);
b = doubleItOpti(a);

...

// Non-optimized version
// Let's assume a is saved in s0
int doubleIt(int myvar) {
  int a = myvar; // we WANT a in s0 !
  a = a * 2;
  return a;
}

// Optimized version
// Choose your registers freely
// Try having the less possible
//   instructions
int doubleItOpti(int myvar) {
  int a = myvar; //
  a = a * 2;
  return a;
}
```

c)
```
int a = 1, b;
b = callA(a);

...

// Functions can be
// optimized at will

int callA(int v1) {
  v1 = v1 * 2;
  return callB(v1);
}

int callB(int v1) {
  v1 = v1 + 12;
  return v1;
}
```

d)
```
int a=1, b=2, c=3,
 d=4, e=5, f=6, g=7,
 h=8, i=9, j=10, res;
res = sum(a,b,c,d,e,f,
  g,h,i,j);

...

int sum(int v1, int v2,
  int v3, int v4, int v5,
  int v6, int v7, int v8,
  int v9, int v10){
  int c;
  c = v1 + v2 + v3 + v4 +
    v5 + v6 + v7 + v8 +
    + v9 + v10;
  return c;
}
```

3.5 Algorithms

3.5.1 Fibonacci par récursivité

La notion de récursivité implique qu'une fonction se rappelle elle-même pour calculer un résultat, de la même manière qu'elle appellerait une autre fonction.

Le but est de calculer la suite de fibonacci d'un nombre entier non-signé selon l'algorithme suivant :

```
unsigned int fibbonacci(unsigned int n){
  if(n == 0){
    return 0;
  } else if(n == 1) {
    return 1;
  } else {
    return (fibbonacci(n-1) + fibbonacci(n-2));
  }
}
```

- Implémenter la fonction donnée
- Tester et valider la fonction
- Quelles sont les valeurs n supportées ?

Il est possible d'accélérer grandement le calcul en utilisant la mémoïsation. Le principe est de réserver de la place mémoire et de stocker les résultats déjà connus pour chaque n calculé et de réutiliser cette valeur directement. RV32IM est conseillé ici.

- Implémenter la fonction donnée
- Tester et valider la fonction
- Comparer les algorithmes

3.5.2 Modulo

Le modulo **%** est une opération qui se pratique sur deux nombres entiers positifs et n'est autre que le reste de la division. Par exemple, 5 divisé par 3 donne 1 (on peut faire passer une fois 3 dans 5), reste 2.
Le modulo d'un nombre par 0 n'est pas défini.
La définition pour les nombres signés diverge selon le langage. Nous ne traitons que le version avec les unsigned.
Le modulo a plusieurs utilités, permettant de plafonner des valeurs, extraire de l'information, calculer une position X et Y à partir d'une valeur X*Y dans un tableau de taille connue ...

- Donner un code permettant d'effectuer cette opération pour n'importe quel entier positif en utilisant le set RV32IM.
- Comment la même chose peut-elle être implémentée dans RV32I ? Décrivez le(s) concept(s).

Le rôle du compilateur est d'optimiser au mieux le code. Si l'opération détectée est un modulo avec une constante étant une puissance de 2 (ex. x % 2, y % 8 ...), une variante n'incluant aucune division est possible.

- Donner cette variante.

> La notion de modulo pour les nombres réels est arrivée avec l'évolution des puissances de calcul et le résultat diverge aussi selon le langage. En C, une fonction spécifique des librairies std est nécessaire, **fmod()**. En Python, cette opération est native. Dans tous le cas, elles sont plus gourmandes en ressources et nécessitent de gérer certains cas spécifiques (NaN, ∞).

3.5.3 Nombre carré

Un nombre mis au carré n'est autre que la multiplication dudit nombre par lui-même. Mais ici, nous ne travaillons qu'avec le set d'instruction RV32I :

- Proposer une fonction capable de calculer $n * n$ par une boucle d'additions.
- Tester avec n = 5, 10 et 20.
- Généraliser la fonction pour pouvoir lui fournir deux paramètres en calculant $i * j$.
- Tester avec i = 5, j = 100. Tester avec i = 100, j = 5. Que constatez-vous ?
- Optimiser la fonction pour que l'ordre des opérandes n'influe pas sur le temps de calcul.

3.5.4 °F -> °C

Nous souhaitons créer une fonction capable de convertir les degrés Celsius en Fahrenheit. Comme les valeurs Fahrenheit varient entre 32 et 1000, une précision au degré près est suffisante.

La formule est simple : $C = (F - 32) * \frac{5}{9}$.

Ce serait pratique si un FPU était disponible, mais il ne supporte que le jeu d'instructions de base RV32I.

Pour contourner ce problème, vous pouvez utiliser quelques astuces, l'algorithme est le suivant :

- Calculer $C = F - 32$.
- Multiplier par 5
- Diviser par 9
  - $\frac{1}{9}$ peut être enregistré par une représentation binaire spéciale

    ```
    +-----+-----+-----+-----+-----+-----+------------+------------+
    | b31 | b30 | b29 | b28 | b27 | ... |         b1 |         b0 |
    +-----+-----+-----+-----+-----+-----+------------+------------+
    | 2^0 | 2^-1| 2^-2| 2^-3| 2^-4| ... |      2^-30 |      2^-31 |
    +-----+-----+-----+-----+-----+-----+------------+------------+
    | 1   | 1/2 | 1/4 | 1/8 | 1/16| ... |1/1737418240|1/2147483648|
    +-----+-----+-----+-----+-----+-----+------------+------------+
    ```

    dans ce cas, il s'agit de $0000_1110_0011_1000_1110_0011_1000_1110_2$.
  - La constante peut être précalculée et vaut $\frac{2^n}{9} + 1$. Plus n est grand, plus la précision est élevée. Le nombre de bits définit la taille maximale de n. Le $+1$ est un arrondi pour la précision perdue.
  - Prenons $n = 16$. Notre nombre magique est donc $magic = \frac{2^n}{9} + 1 = \frac{65536}{9} + 1 = 7282$.
  - Il faut multiplier la valeur par ce nombre magique et ensuite le diviser par 2^n. Dans le cas $n = 16 \rightarrow \frac{1}{65536}$.

Pour simplifier le travail, plusieurs hypothèses sont faites :

- Le nombre magique et la température en Fahrenheit sont toujours positifs.
- La taille de la plus grande multiplication est :

$$\begin{aligned} nbBits_{max_{fahrenheit}} + nbBits_{mult5} + nbBits_{magicNumber} &= \\ 10\textit{(max. 1000-32)} + 3 + (n - nbBits_{div9} + 1) &= \\ \}.10 + 3 + (16 - 4 + 1) &= \\ &= 26bits \end{aligned}$$

Elle ne dépasse jamais 32 bits (pour n $<$ 23). crivez le code correspondant.

Testez et optimisez la fonction :

- Tester avec différentes valeurs de Fahrenheit.
- Tester avec plusieurs valeurs pour n (16, 18, 20). *N'oubliez pas de recalculer le nombre magique.*
- Tester la fonction avec $n = [22, 23]$, pour $F = [100, 400, 1000]$.

4 Assembler / Disassembler

L'assembleur et désassembleur RISC-V utilisé dans ce labo peut être trouvé dans le repo sous `heirv32_sc/heirv32-asm/` pour Windows & Linux amd64 et macOS ARM64. Cet outil vous permet de transformer le code assembleur RV32i ou HEIRV32 en code binaire (Assembly) et de retransformer le code binaire en code assembleur (Disassembly).

```
$ heirv32-asm --help

usage: HEIRV32-ASM
    Assembler/Disassembler to support HEIRV32 ISA,
    See help below for usage : it auto-detects if the file is binary like (=> will disassemble)
    or contains ASM instructions (=> will assemble). If no switches are given, will open a GUI
    to select the file and run with '-t = 'phb'' argument.

    Usage:
    heirv32 -f <ASMfile> [-t="phb"] [-i="HEIRV32" | "RV32I"]
    heirv32 -s <string> [(-t="hb" -of <outputPathName>)] [-i="HEIRV32" | "RV32I"]
    heirv32 -g [-t="phb"] [-i="HEIRV32" | "RV32I"]
    heirv32 -i <ISA>
    heirv32 -h | --help
    heirv32 [-g -t="phb" -i="HEIRV32"]

    Options:
    -h --help     Show this screen.
    -f            Input ASM file.
    -s            Input string.
    -g            Open GUI to select file to convert.
    -i            ISA to use ('HEIRV32' or 'RV32I' supported), default 'HEIRV32'.
                  Can be used alone to log the ISA specs.

    -t            Output type ('p' - print, 'h' - BRAM file, 'b' - bin File), default 'p'.
    -of           Output file path whe convertin an input string ('-s'), if 'h' and/or 'b'
                  are used for '-t'.
```

4.1 `HEIRV32-asm` Commandes

Assemblage ou désassemblage automatique en fonction des fichiers d'entrée

4.1.1 Assembleur vers binaire

```
$HEIRV32-ASM_1.1.1_Darwin_ARM -f ./tests/tassembly.s -t p

Generating file with output type p

File: ./tests/tassembly.s
Conversion ongoing
 * Using the ASSEMBLER
 * Found a value in code : 1500 at address 88 - do not let your code try to run it as
 ↪  an instruction !
 * Found a value in code : 1 at address 92 - do not let your code try to run it as an
 ↪  instruction !
 * Found a value in code : 255 at address 96 - do not let your code try to run it as
 ↪  an instruction !
 ** Found data / instructions: 25

------ p : Printing instructions list and write to file ------
 * Output file: _syn.txt
0x0000: addi x2 x0 5      => 0b00000000'01010000'00000001'00010011 => 0x00500113
0x0004: addi x3 x0 12     => 0b00000000'11000000'00000001'10010011 => 0x00c00193
0x0008: addi x7 x3 -9     => 0b11111111'01110001'10000011'10010011 => 0xff718393
0x000c: or   x4 x7 x2     => 0b00000000'00100011'11100010'00110011 => 0x0023e233
```

```
0x0010: and  x5 x3 x4     => 0b00000000'01000001'11110010'10110011 => 0x0041f2b3
0x0014: add  x5 x5 x4     => 0b00000000'01000010'10000010'10110011 => 0x004282b3
0x0018: beq  x5 x7 end    => 0b00000010'01110010'10001000'01100011 => 0x02728863
0x001c: slt  x4 x3 x4     => 0b00000000'01000001'10100010'00110011 => 0x0041a233
0x0020: beq  x4 x0 around => 0b00000000'00000010'00000100'01100011 => 0x00020463
0x0024: addi x5 x0 0      => 0b00000000'00000000'00000010'10010011 => 0x00000293
0x0028: slt  x4 x7 x2     => 0b00000000'00100011'10100010'00110011 => 0x0023a233
0x002c: add  x7 x4 x5     => 0b00000000'01010010'00000011'10110011 => 0x005203b3
0x0030: sub  x7 x7 x2     => 0b01000000'00100011'10000011'10110011 => 0x402383b3
0x0034: sw   x7 84(x3)    => 0b00000100'01110001'10101010'00100011 => 0x0471aa23
0x0038: lw   x2 88(x0)    => 0b00000101'10000000'00100001'00000011 => 0x05802103
0x003c: add  x9 x2 x5     => 0b00000000'01010001'00000100'10110011 => 0x005104b3
0x0040: jal  x3 end       => 0b00000000'10000000'00000001'11101111 => 0x008001ef
0x0044: addi x2 x0 1      => 0b00000000'00010000'00000001'00010011 => 0x00100113
0x0048: add  x2 x2 x9     => 0b00000000'10010001'00000001'00110011 => 0x00910133
0x004c: sw   x2 0x20(x3)  => 0b00000010'00100001'10100000'00100011 => 0x0221a023
0x0050: beq  x2 x2 main   => 0b11111010'00100001'00001000'11100011 => 0xfa2108e3
0x0054: jal  x3 main      => 0b11111010'11011111'11110001'11101111 => 0xfadff1ef
0x0058: 1500              => 0b00000000'00000000'00000101'11011100 => 0x000005dc
0x005c: 0b1               => 0b00000000'00000000'00000000'00000001 => 0x00000001
0x0060: 0xff              => 0b00000000'00000000'00000000'11111111 => 0x000000ff
```

4.1.2 Binaire vers assembleur

```
./dist/HEIRV32-ASM_1.1.1_Darwin_ARM -f ./tests/tassembly.txt -t p

Generating file with output type p

File: ./tests/tassembly.txt
Conversion ongoing
 * Using the DISASSEMBLER
 * Finding instructions
  ** 00000000000000000000010111011100 no match - assuming value
  ** 00000000000000000000000000000001 no match - assuming value
  ** 00000000000000000000000011111111 no match - assuming value
 * Recomposing rd, rs and immediates
 * Adding labels
['addi x2 x0 5', 'addi x3 x0 12', 'addi x7 x3 -9', 'or x4 x7 x2', 'and x5 x3 x4', 'add
↪  x5 x5 x4', 'beq x5 x7 48', 'slt x4 x3 x4', 'beq x4 x0 8', 'addi x5 x0 0', 'slt x4
↪  x7 x2', 'add x7 x4 x5', 'sub x7 x7 x2', 'sw x7 84(x3)', 'lw x2 88(x0)', 'add x9 x2
↪  x5', 'jal x3 8', 'addi x2 x0 1', 'add x2 x2 x9', 'sw x2 32(x3)', 'beq x2 x2 -80',
↪  'jal x3 -84', '(value) 1500', '(value) 1', '(value) 255']
 Modifying beq x2 x2 -80 with label 0 - instr nb. 20
 Modifying jal x3 -84 with label 0 - instr nb. 21
 Modifying beq x4 x0 8 with label 1 - instr nb. 8
 Modifying beq x5 x7 48 with label 2 - instr nb. 6
 Modifying jal x3 8 with label 2 - instr nb. 16
 * Recomposed labels / instructions: 28

------ p : Printing instructions list ------
lb0:
addi x2 x0 5
addi x3 x0 12
addi x7 x3 -9
or x4 x7 x2
and x5 x3 x4
add x5 x5 x4
beq x5 x7 lb2
slt x4 x3 x4
beq x4 x0 lb1
addi x5 x0 0

lb1:
slt x4 x7 x2
add x7 x4 x5
```

```
sub x7 x7 x2
sw x7 84(x3)
lw x2 88(x0)
add x9 x2 x5
jal x3 lb2
addi x2 x0 1

lb2:
add x2 x2 x9
sw x2 32(x3)
beq x2 x2 lb0
jal x3 lb0
(value) 1500
(value) 1
(value) 255
```

4.1.3 Prise en main

- Enregistrer le code suivant sous un fichier **.s** :

```
myLabel:
  li   x20 0b11111111111
  add  x20 x20 x20
  add  x20 x20 x20
  add  x20 x20 x20
  addi x3 x0 0
begin:
  mv   x1 x0
a: # Should be useful for something
  slt  x2 x1 x20
  beq  x2 x0 subing
  addi x1 x1 1
  beq  x0 x0 a
subing:
  addi x3 x3 -1
  beq  x3 x0 test
label_wo_beq:
  addi x30 x0 0xCC
  jal  ra begin
test:
  addi x30 x0 0b00110011
  jal  begin
l6: # This is very important
# But won't always save you
  jal  ra myLabel
```

- Compilez-le à l'aide de HEIRV32-ASM avec les sorties **print** et **bin** activées :
  `./dist/HEIRV32-ASM_1.1.1_xxx -i RV32I -f ./my/file/disassembly-01.s -t pb`.
  Un fichier **.txt** est créé, qui est une version hexadécimale lisible par les humains du fichier **.bin** généré.
- Maintenant, décompilez-le fichier **.txt** de la même manière pour recréer un fichier assembleur :
  `./dist/HEIRV32-ASM_1.1.1_xxx -i RV32I -f ./my/file/assembled-01.txt -t pb`.
  Cela devrait créer un fichier ***_disassembly.s**.
- Qu'est-ce qui change entre les deux codes ?
- Au fait, à quoi sert-il ?

4.2 Fonction main

Dans le programme Ripes montré sous 3.1, compilez les codes suivants et cherchez dans le code assembleur les instructions assembleur correspondantes. Que signifient les différentes

instructions dans `main:` ?

```
void main() {
  int a = 0;
}
```

```
void main() {
  while(1) {
    int a = 0;
  }
}
```

4.3 Reverse Engineering

Vous travaillez sur un projet en binôme sur un processeur RISC-V quelque peu modifié.

En effet, pour se simplifier la vie et communiquer avec le monde extérieur, ce dernier peut lire l'état de 4 boutons reliés au processeur en lisant l'adresse mémoire **0xf0000000**, et allumer 8 leds en écrivant aux adresses mémoire **0xf0000004** (led 0) à **0xf0000020** (led 7) :

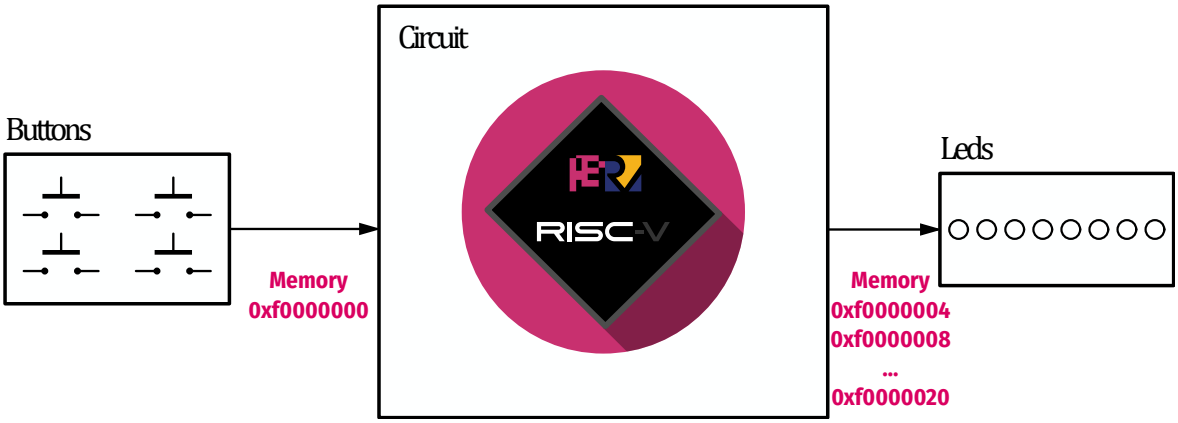


FIGURE 6 – Processeur RISC-V modifié

Le premier laboratoire consistait à faire clignoter les leds selon les boutons appuyés :

- Aucun bouton : les 8 leds sont allumées selon le pattern 0xAA
- Le bouton 0 est à '1' : les 8 leds clignotent à une fréquence raisonnable (le clignotement est visible)

Pour simplifier l'accès au bouton et aux leds, le code suivant vous était donné :

```
setup:
    # led is sw xx, offBy4(x30) with xx loaded as 0x00rrggbb
    li x30, 0xf0000004 # base address for leds
    li x31, 0xf0000000 # base address for buttons, one register

    # DO NOT MODIFY x30 (t5) and X31 (t6) !!!

get_btns: # return buttons value in a0
  lw a0, 0(x31)
  jr ra

set_leds: # pass a 8 bits which if bit(n) = '1', led(n) is on
  li t0, 8   # loop the leds
  mv t3, x30 # mem position
  addi t3, t3, 28 # leds display is reversed, so stock reverted

  leds_loop:
  beq x0, t0, leds_end
```

```
andi t1, a0, 1
# if 1, lights corresponding led
beq t1, x0, isoff

ison:
li t2, 0x00ff00ff
j leds_loop_end

isoff:
mv t2, x0

leds_loop_end:
sw t2, 0(t3) # save led value
srli a0, a0, 1  # shift right leds value
addi t0, t0, -1 # decrement loop
addi t3, t3, -4  # add memory pos
j leds_loop

leds_end:
jr ra
```

Malheureusement, votre collègue est absent aujourd'hui et ne vous a pas laissé une copie du code déjà écrit. En plus, il y'avait quelques bugs :

- Aucun bouton : les leds s'allument selon le pattern 0x2A
- Le bouton 0 est à '1' : seules 7 leds clignotent, et à une fréquence qui semble bien trop élevée
- Un des autres bouton est appuyé : même comportement que si le bouton 0 était appuyé

Il semblerait que vous ayez mal calculé la boucle d'attente du clignotement. De plus, les leds ne suivent pas les patterns énoncés et à une trop grande vitesse. Un problème matériel peut être exclu.

Vous vous souvenez que le dernier code écrit avait été flashé sur votre board RISC-V et, vu que le chip n'avait pas été protégé contre la relecture, vous arrivez à l'extraire par accès au bus JTAG, ce qui vous a donné :

```
f0000f37
00400f13
f0000fb7
00000f93
00000413
040000ef
02a00463
00900663
fff48493
ff1ff06f
06400493
00800663
00000413
0140006f
0bf00413
00c0006f
02a00413
00000493
00040513
010000ef
fc5ff06f
000fa503
00008067
00800293
000f0e13
01ce0e13
02500663
```

```
00157313
00030663
00fff3b7
fff00393
0080006f
00000393
007e2023
00155513
fff28293
ffce0e13
fd5ff06f
00008067
```

- Extrayez votre code
- Identifiez et séparez le code donné du reste
- Comprendre l'utilisation des fonctions données

Pour simuler le système, sous Ripes, ouvrez l'onglet I/O :

- Double-cliquez sur LED Matrix, sélectionnez le widget créé et configurez-le tel que :

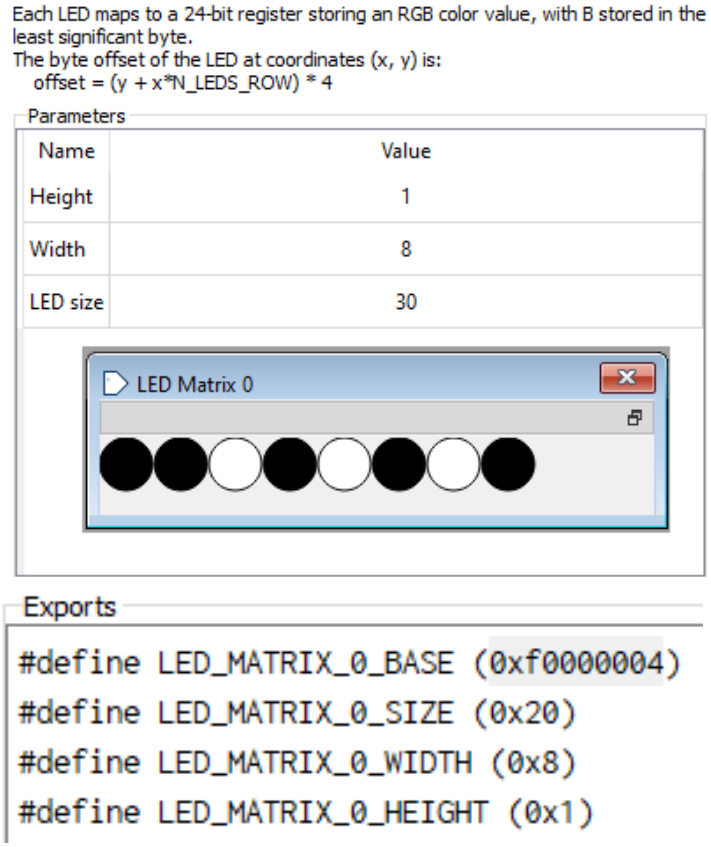


FIGURE 7 – Ripes - configuration des leds

Contrôlez que **Exports** contienne les mêmes informations que sur l'image donné après réglage du module.

- Double-cliquez sur Switches, sélectionnez le widget créé et configurez-le tel que :

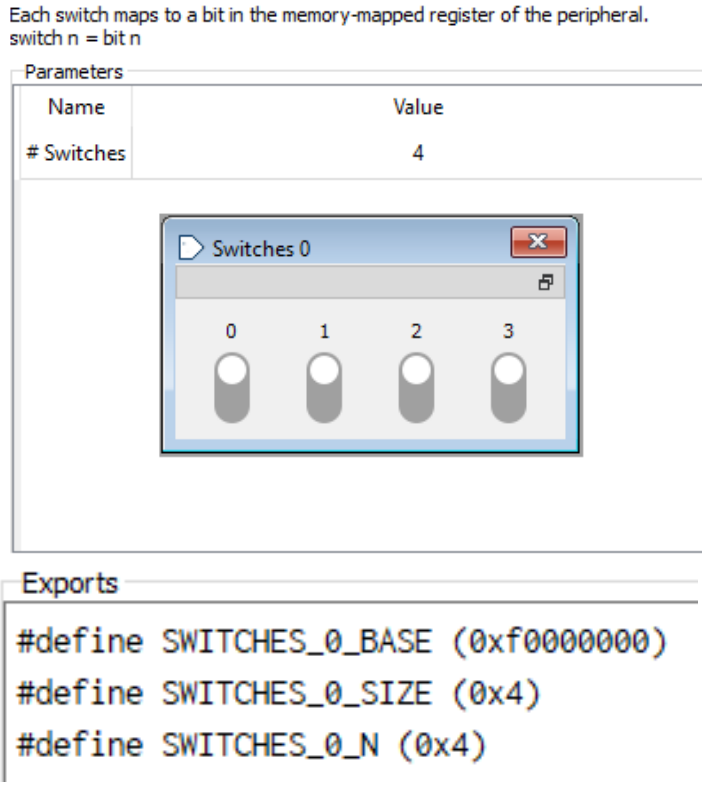


FIGURE 8 – Ripes - configuration des interrupteurs

Contrôlez que **Exports** contienne les mêmes informations que sur l'image donné après réglage du module.

Chargez votre code assembleur dans **Editor**, appuyez sur F8 (ou l'icône ») et constatez que le circuit fonctionne selon les problèmes énoncés en interagissant sous **I/O** avec les interrupteurs.

- Fixez les problèmes cités
- Testez votre circuit sur Ripes

La clock ne pouvant être réglée de manière précise, la fréquence de clignotement des leds se fait sans calcul. Corriger le code pour obtenir environ 2 Hz.

Ce que vous venez d'accomplir s'apparente à du hardware hacking : un code est dumpé (copié), analysé, modifié, puis réinjecté dans un système pour l'adapter à son envie.
Bien sûr, il n'est pas toujours si simple de copier et réinjecter un code. Des méthodes parfois exotiques ont vu le jour (voir le Kamikaze Hack de la Xbox 360).

Acronymes

FPU **F**loating **P**oint **U**nit. 11

ISA **I**nstruction **S**et **A**rchitecture. 1

LSB **L**east **S**ignifiant **B**it. 5

RISC-V **R**educed **i**nstruction **s**et **c**omputer **V**. 1, 3, 6, 12, 15, 16